

CLASS 9a

FINE-TUNING

What, Why & Continued Pre-Training from Scratch

The Journey So Far

Pre-training → The model learns language

Prompting → We talk to the model

RAG → We give it external knowledge

Agents → We give it tools

Fine-tuning → We change the model itself

What Is Fine-Tuning?

Pre-training

Learn language from the internet

trillions of tokens, months, \$\$\$\$\$

Fine-tuning

Adapt that knowledge for your needs

thousands of tokens, hours, \$

Same mechanism. Different scale. Different data.

Like a university graduate who already knows how to learn —
you're not teaching them from scratch,
you're training them for a specific job.

Why Fine-Tune?

Prompting doesn't work when:

- ▶ The model doesn't know your domain's language
- ▶ You need a specific style/format consistently
- ▶ You need the model smaller/faster for production
- ▶ You want behavior the prompt can't enforce

RAG doesn't work when:

- ▶ The issue isn't missing knowledge, it's wrong behavior
- ▶ Latency matters (no retrieval step)
- ▶ The model can't understand what it retrieves

Key insight: RAG adds knowledge. Fine-tuning changes behavior and style.
They solve different problems. Best production systems combine both.

The Decision Framework

"Model doesn't KNOW something"



RAG or CPT

"Model doesn't DO something well"



SFT

"Model does it but not how humans want"



RLHF / DPO

"Model needs to reason better"



RLVR

Best systems → Combine all of them

The Full Post-Training Pipeline

Base Model (next-token predictor)

- └ CPT (Continued Pre-Training) ← raw text, same loss
- └ SFT (Supervised Fine-Tuning) ← instruction-response pairs
- └ RLHF (RL from Human Feedback) ← human preferences
 - └ DPO (Direct Preference Opt.) ← simpler alternative
- └ RLVR (RL with Verifiable Rewards) ← math/code/logic

Today: CPT **Next:** SFT **Then:** RLHF/DPO → RLVR

PART 1

WHAT'S ACTUALLY HAPPENING INSIDE

What's Inside a Model?

A transformer is a stack of layers. Each layer has:

ATTENTION BLOCK

`Q_proj (W_q)`

`K_proj (W_k)`

`V_proj (W_v)`

`O_proj (W_o)`

"Which tokens to pay attention to"

FFN BLOCK

`gate_proj (W_g)`

`up_proj (W_up)`

`down_proj (W_dn)`

"What to do with the information"

+ `embed_tokens (input)`

+ `lm_head (output)`

These are all WEIGHT MATRICES. Giant grids of numbers.

Full Fine-Tuning — The Naive Approach

Full fine-tuning = update EVERY weight in EVERY matrix

Forward: input → model → prediction

Loss: how wrong was the prediction?

Backward: compute gradient for EVERY parameter

Update: nudge EVERY parameter to reduce loss

Simple. Effective. The "obvious" approach.

The Memory Math (Why Full Fine-Tuning Breaks)

To TRAIN a model, you need in GPU memory:

- | | | |
|---------------------|---------------|--------------------------|
| 1. Model weights | 1× model size | |
| 2. Gradients | 1× model size | (one gradient per param) |
| 3. Optimizer states | 2× model size | (AdamW stores m and v) |
-

Total: ~4× model size (minimum!)
+ activations, batch data → often 6-8×

SmolLM-135M fp16: $135\text{M} \times 2\text{B} = 270\text{ MB}$ → ~1-2 GB to train ✓

LLaMA-3 8B fp16: $8\text{B} \times 2\text{B} = 16\text{ GB}$ → ~64-128 GB △

LLaMA-3 70B fp16: $70\text{B} \times 2\text{B} = 140\text{ GB}$ → ~560+ GB ✗

A100 GPU: 80 GB. Your laptop: 8-16 GB.

Full Fine-Tuning Also Has a Quality Problem

Problem 1: Memory

(we just saw this)

Problem 2: Overfitting

1000 SEC filings (~5M tokens) vs 8 BILLION parameters
= massive overfitting. Memorizes instead of learning.

Problem 3: Catastrophic Forgetting

Every weight gets updated. General knowledge gets overwritten. No "safe zone" of preserved knowledge.

Problem 4: Storage & Serving

Each fine-tuned model = full copy of all weights.
5 variants of LLaMA-70B = $5 \times 140 \text{ GB} = 700 \text{ GB}$.

We Need a Better Approach

- ✓ Train on a single GPU (or even a laptop)
- ✓ Don't overfit on small datasets
- ✓ Preserve general knowledge
- ✓ Store just the changes, not the whole model
- ✓ No inference slowdown

**What if we could freeze the original model
and only train a TINY set of new parameters?**

LoRA — The Key Insight

Observation from the paper (Hu et al., 2021):

"When you fine-tune a large model, the weight CHANGES have very low intrinsic dimensionality."

Translation:

- ▶ The update matrix ΔW is LOW-RANK.
- ▶ Most of ΔW is redundant.
- ▶ You can represent the same update with far fewer numbers.

Like a photo of the sky — mostly one color. You don't need 10M unique pixels to represent it. The weight updates have the same compressibility.

What Does "Low-Rank" Mean? (Intuition)

Rank 4096: Full matrix. $4096 \times 4096 = 16.7\text{M}$ numbers. Unique.

Rank 1: Just one pattern repeated and scaled.

Rank 32: 32 independent patterns combined.

Fine-tuning updates tend to be rank 32-ish,
even for a 4096×4096 matrix.

Analogy: The Choir

A choir of 100 people singing.

But only ~4 are singing unique parts.

The other 96 are harmonizing/doubling.

You only need to record the 4 unique voices.

LoRA — The Mechanism

Original weight matrix W : $(4096 \times 4096) = 16.7\text{M}$ params

Instead of learning the full update ΔW :

$$\Delta W = B \times A$$

A : $(4096 \times 32) = 131\text{K}$ params "down-project to rank"

B : $(32 \times 4096) = 131\text{K}$ params "up-project back"

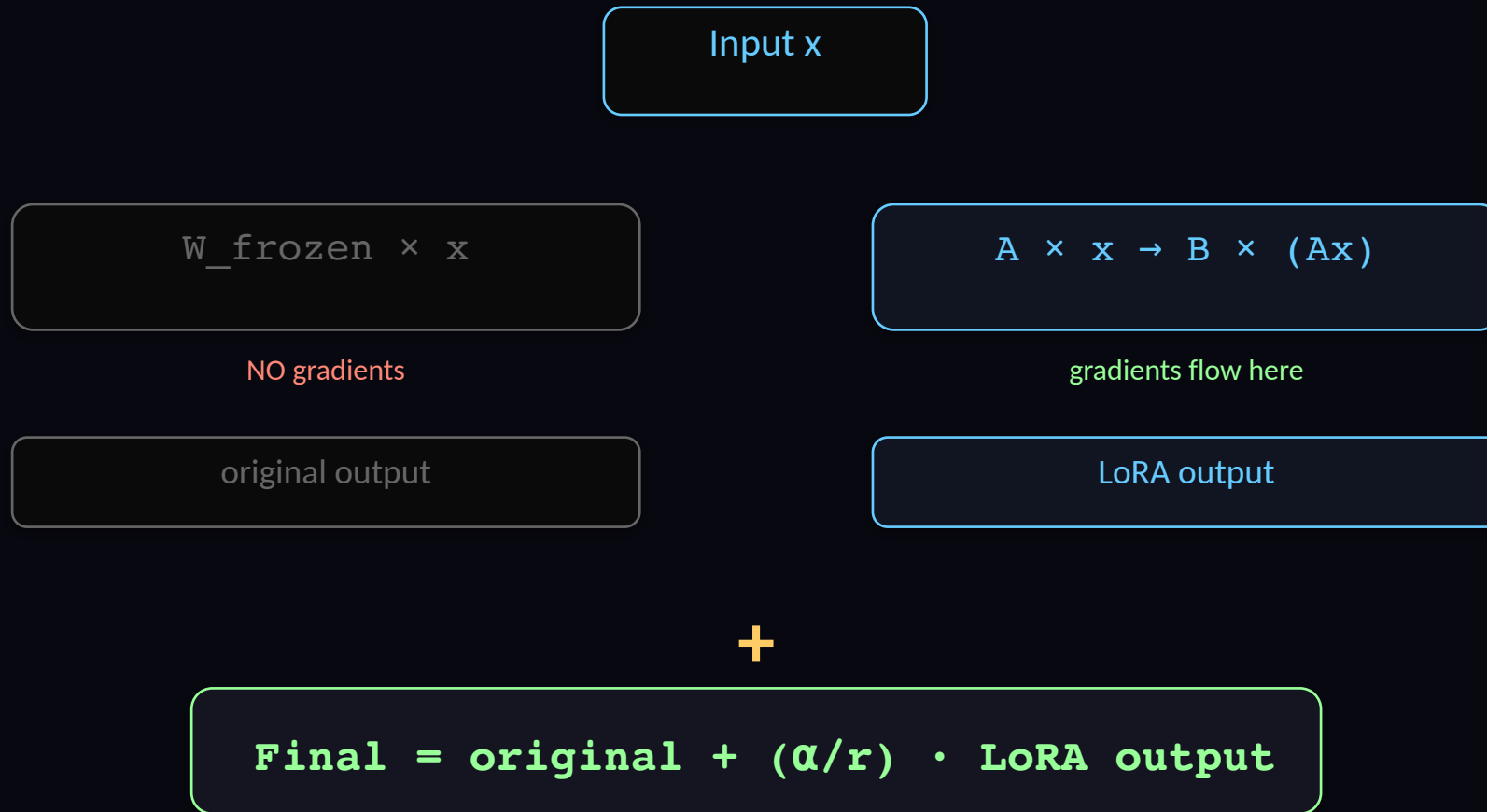
Total: 262K params vs 16.7M (64× fewer!)

W is FROZEN. Only A and B are trained.

Final output: $W_{\text{frozen}} \cdot x + (\alpha/r) \cdot B \cdot A \cdot x$

Freeze the original. Train only the tiny adapters.

LoRA Visually



After training: merge! $W_{\text{new}} = W + (\alpha/r) \cdot BA$
One matrix. Zero added inference cost.

The Memory Savings (Concrete)

Full Fine-Tuning LLaMA-3 8B

Weights: 16 GB

Gradients: 16 GB

Optimizer: 32 GB

Total: ~64+ GB ← needs A100

LoRA (rank 32) on LLaMA-3 8B

Frozen weights: 16 GB → 4-bit → 4 GB

LoRA params: ~40 MB

Gradients: ~40 MB

Optimizer: ~80 MB

Total: ~4.2 GB ← fits on laptop!

Saved adapter file: ~80 MB (vs 16 GB for full model)

5 customers, 5 fine-tunes?

400 MB of adapters sharing one base model vs 80 GB of full copies.

LoRA Hyperparameters

r (rank)

Size of the bottleneck. Higher = more capacity, more memory.
Common: 8, 16, 32, 64

α (alpha)

Scaling factor. Controls how much LoRA affects the output.
Rule of thumb: $\alpha = r$ (so scaling = 1)

target_modules

Which weight matrices to add LoRA to.
Minimum: q_proj, v_proj | Recommended: all attention + all FFN
CPT special: + embed_tokens, lm_head

dropout

Usually 0 (Unsloth optimizes for this)

Why CPT Needs `embed_tokens` and `lm_head`

`embed_tokens`: token ID $\rightarrow$ vector "What does this word mean?"
`lm_head`: vector $\rightarrow$ token ID "What word should come next?"

SFT

Model already knows the vocabulary.

Teaching it to use words differently.

→ Don't need to touch embeddings.

CPT

Model is learning NEW domain language.

"EBITDA", "10-K", "diluted EPS",
"subordinated debentures"

→ **MUST** update `embed_tokens` + `lm_head`

**Most common CPT mistake: copying an SFT LoRA config
and wondering why domain adaptation doesn't work.**

Quantization — Loading in 4-bit

Float32: 32 bits/param → $135\text{M} \times 4\text{B} = 540 \text{ MB}$

Float16: 16 bits/param → $135\text{M} \times 2\text{B} = 270 \text{ MB}$

Int8: 8 bits/param → $135\text{M} \times 1\text{B} = 135 \text{ MB}$

NF4: 4 bits/param → $135\text{M} \times 0.5\text{B} = 67 \text{ MB}$

QLoRA = Quantized base (4-bit) + LoRA adapters (16-bit)

The frozen weights are compressed. Only the tiny LoRA adapters are kept in full precision.

NF4 is designed for neural network weight distributions — almost no quality loss

What Unsloth Actually Does

Unsloth is NOT a different training algorithm. It's the SAME math, made faster.

What it does:

- ▶ Custom CUDA kernels for attention + LoRA (2× speed)
- ▶ Fused operations (fewer memory round-trips)
- ▶ Smart gradient checkpointing (50% less VRAM)
- ▶ Automatic mixed precision (bf16/fp16)
- ▶ Optimized packing (no wasted padding tokens)

What it doesn't do:

- ▶ Change the training objective
- ▶ Change the model architecture
- ▶ Do anything you couldn't do with raw HF + PEFT + TRL

Like NumPy vs hand-written Python matrix multiplication.
Same math, wildly different speed. If Unsloth vanished, you'd use HF Transformers + PEFT + TRL.

Gradient Checkpointing (How Unsloth Saves 50% VRAM)

Normal training: store ALL activations

Layer 1 output → [STORED]
Layer 2 output → [STORED]
Layer 3 output → [STORED]
...
Layer 24 output → [STORED]

Memory grows with depth!

Gradient checkpointing: store SOME

Layer 1 output → [STORED]
Layer 2 output → [RECOMPUTE when needed]
Layer 3 output → [STORED]
Layer 4 output → [RECOMPUTE when needed]
...

Tradeoff: ~30% slower training, ~50% less VRAM.
Unsloth's version: smarter about WHICH layers to checkpoint.

This is what `use_gradient_checkpointing='unsloth'` does in our config.

The Training Loop Internals

1. **TOKENIZE** text → token IDs

2. **FORWARD** tokens through model → logits (predictions)

3. **LOSS** cross-entropy between predicted and actual next token

4. **BACKWARD** compute gradients (only for LoRA params!)

5. **UPDATE** optimizer step (AdamW) on LoRA params only

6. **REPEAT**

Same loop for pre-training, CPT, and SFT.
The only difference is the DATA.

Key Training Concepts

Packing

Concatenate short texts into one sequence.

Without: [text1 + PAD PAD PAD] With: [text1 | text2 | text3] ← no waste

Gradient Accumulation

Simulate larger batch sizes.

Batch size 32, accumulation 2 = effective batch 64

Warmup

Start with tiny learning rate, ramp up.

Prevents early training from destroying weights.

Early Stopping

Monitor eval loss. Stop if it stops improving.

Prevents overfitting on small datasets.

PART 2

THE ENEMY — CATASTROPHIC FORGETTING

What Happens When You Only Train on Domain Data

Before CPT:

"The cat sat on the" → "mat" ✓ general English

"Revenue increased by" → random garbage ✗ no finance

After CPT (no mixing):

"Revenue increased by" → "12% year over year" ✓ finance!

"The cat sat on the" → "balance sheet" ✗ forgot English

The weights that encoded general language get overwritten.

Why Forgetting Happens (The Mechanics)

Gradient updates are GREEDY.

At each step, the optimizer only cares about reducing loss on the CURRENT batch.

If every batch is SEC filings:

- Gradients always point toward "be better at finance"
- Gradients never point toward "stay good at general text"
- General knowledge slowly erodes

**The model doesn't "choose" to forget.
The optimizer just never gets a signal to remember.**

Solutions to Forgetting

1. Data mixing

80% domain + 20% general data

2. Low learning rate

Don't move weights too far from base (1e-5 full, 2e-4 LoRA)

3. LoRA itself

Freezing base weights IS forgetting prevention!

4. Short training

Watch eval loss — when it plateaus, stop.

5. Embedding LR

Lower LR for embed_tokens/lm_head (10% of main).
These layers affect ALL tokens, not just domain.

PART 3

HANDS-ON — CPT WITH SEC 10-K FILINGS

The Plan

- 1 Load SmolLM-135M → try financial text → watch it fail
- 2 Get SEC 10-K filings from HuggingFace (PleIAs/SEC)
- 3 Clean & chunk the data
- 4 CPT with Unsloth + LoRA
- 5 Try financial text again → see the improvement
- 6 Measure: perplexity before vs after
- 7 Experiment: with vs without data mixing

What Are 10-K Filings?

10-K = Annual report every public US company files with the SEC

- ▶ Business description
- ▶ Risk factors
- ▶ Financial statements
- ▶ Management discussion & analysis (MD&A)

Language style:

"The Company recognized impairment charges of \$142 million related to goodwill associated with the North America reporting unit during the fiscal year ended March 31, 2024."

SmolLM has never seen this kind of text at scale.

Data Preparation

Raw 10-K filing

|

— Strip boilerplate (legal disclaimers, page numbers)

— Remove reference/exhibit sections

— Word-level chunking (256 words, 20% overlap)

|

↓

JSONL: {"text": "chunk of financial text..."}

Train/Val split: hold out ~50 filings for evaluation

JSONL format is what Unsloth's SFTTrainer expects for text completion tasks.

The Training Config (Explained)

```
# LoRA Config
r = 32                                # Higher rank for CPT
target_modules = [
    "q_proj", "k_proj", "v_proj",    # Attention matrices
    "o_proj",                        # Output projection
    "gate_proj", "up_proj",          # FFN layers (knowledge lives here!)
    "down_proj",
    "embed_tokens", "lm_head"        # CPT-specific: new vocabulary
]
lora_alpha = 32                       # = rank, scaling factor = 1

# Training
learning_rate = 2e-4                 # Standard for LoRA
embedding_learning_rate = 2e-5       # 10× lower for embeddings
packing = True                       # No wasted padding
warmup_steps = 100                   # Gentle start
```

Target ALL linear layers including FFN — that's where factual knowledge is stored

Training — What to Watch

Good training run:

- ▶ Train loss steadily decreasing
- ▶ Eval loss tracking train loss (not diverging)
- ▶ Eval loss plateaus → stop (early stopping)

Bad signs:

- ▶ Eval loss up while train loss drops → overfitting
- ▶ Train loss spikes → learning rate too high
- ▶ Loss stuck → LR too low or data too small

Perplexity = $e^{(\text{loss})}$

Lower = better. Means model is less "surprised" by the text.

The eval loss is your north star.

Results Comparison

Prompt: "The company reported total revenue of"

Base SmolLM:

"the most important thing is to be able to get a good understanding of the world"

After CPT:

"approximately \$4.2 billion for the fiscal year ended December 31, 2023, representing an increase of 12%"

Perplexity on SEC test set:

Base: ~180

CPT: ~45

4× lower perplexity — from confused to fluent in financial language

NEFTune — A Subtle Trick

```
NEFTune: Add random noise to embeddings during training
```

```
input_embeds = input_embeds +  $\alpha$  * random_noise
```

Why?

- ▶ Acts as regularization (like dropout but for embeddings)
- ▶ Prevents the model from overfitting to exact token sequences
- ▶ Consistent improvements, especially on small datasets

Used in full training mode (not LoRA, since LoRA already regularizes by constraining the update space)

The Forgetting Experiment

Run A:

100% SEC filings

No general data mixed in

Run B:

80% SEC + 20% general text

(scientific_papers)

Evaluate on:

- ▶ SEC test set (domain performance)
- ▶ General text (wikitext or similar)

Expected:

Run A: Great on SEC, degraded on general

Run B: Slightly worse on SEC, maintained general

PART 4

THE LIMIT OF CPT

CPT Made It Smarter, Not Helpful

Prompt: "What was Apple's revenue in fiscal year 2023?"

After CPT:

"The company's revenue was \$383.3 billion for the fiscal year ended September 30, 2023, representing a decrease of 2.8% compared to the prior fiscal year revenue of \$394.3..."

It continued the text. It didn't ANSWER the question.

CPT teaches the model to **SOUND** like a domain.

SFT teaches the model to **RESPOND** to instructions.

What's Coming Next

Next Session (9b)

SFT — Teaching the Model to Follow Instructions

Instruction-response dataset format (chat templates)

Training on financial Q&A pairs

The model goes from "autocompleter" to "assistant"

Merge adapters for deployment

After That (9c)

RLHF, DPO & RLVR — Teaching the Model What "Good" Means

Human preference alignment

Direct Preference Optimization

Verifiable rewards for reasoning

Today: what the model knows. Next: what it does. Then: what it values.

Key Takeaways

1. Fine-tuning modifies the model itself (unlike prompting/RAG)

2. LoRA: freeze base, train tiny adapter matrices (64× fewer params)

3. CPT = same loss as pre-training, different (domain) data

4. For CPT: include embed_tokens + lm_head in LoRA targets

5. Catastrophic forgetting is real — mix in general data

6. Unsloth = same math, faster execution

7. CPT teaches domain language, NOT instruction following

Fine-tuning is not magic — it's gradient descent on different data.